

Mid-Square hashing

Last Updated : 20 Mar, 2024



Mid-Square hashing is a hashing technique in which unique keys are generated. In this technique, a seed value is taken and it is squared. Then, some digits from the middle are extracted. These extracted digits form a number which is taken as the new seed. This technique can generate keys with high randomness if a big enough seed value is taken. However, it has a limitation. As the seed is squared, if a 6-digit number is taken, then the square will have 12-digits. This exceeds the range of int data type. So, overflow must be taken care of. In case of overflow, use long long int data type or use [string as multiplication](#) if overflow still occurs. The chances of a collision in mid-square hashing are low, not obsolete. So, in the chances, if a collision occurs, it is handled using some [hash map](#). **Example:**

*Suppose a 4-digit seed is taken. seed = 4765 Hence, square of seed is = $4765 * 4765 = 22705225$ Now, from this 8-digit number, any four digits are extracted (Say, the middle four). So, the new seed value becomes seed = 7052 Now, square of this new seed is = $7052 * 7052 = 49730704$ Again, the same set of 4-digits is extracted. So, the new seed value becomes seed = 7307 This process is repeated as many times as a key is required.*

Mid square technique takes a certain number of digits from the square of a number. This number extracted is a pseudo-random number, which can be used as a key for hashing. **Algorithm:**

1. Choose a seed value. This is an important step as for same seed value, the same sequence of random numbers is generated.
2. Take the square of the seed value and update seed by a certain number of digits that occur in the square. **Note:** The larger is the number of digits, larger will be the randomness.
3. Return the key.

Below is the implementation of above algorithm:

```
// C++ program to illustrate the
// mid-square hashing technique
#include <ctime>
#include <iostream>

using namespace std;

// Returns a seed value based on current system time.
long long int newTime()
{
    // Acquiring number of seconds
    // passed from Jan 1, 1970.
    time_t t = time(NULL);

    // Converting the time to year, month,
    // day, hour, minute, and second.
    struct tm* tm = localtime(&t);
    long long int x;

    // Applying a certain logic to get
    // required number of digits. It may vary.
    x = (tm->tm_hour) * 10000000 + (tm->tm_min) * 100000
        + (tm->tm_sec) * 1000 + (tm->tm_mday) * 10 + (tm-
>tm_year);

    // Return the calculated number.
    return x;
}

// Returns a random 8-digit key.
long int getKey()
{
    // Taking the key from system time.
    // returns a 8-digit seed value.
    static long long int key = newTime();

    // Squaring the key.
    key = key * key;
```

```

// Extracting required number of digits ( here, 8 ).
if (key < 1000000000000000) {
    key = key / 10000;
    key = key % 100000000;
}
else {
    key = key / 10000;
    key = key % 100000000;
}

// Returning the key value.
return key;
}

// Driver Code
int main()
{
    // get the first key
    std::cout << getKey() << endl;

    // get the second key
    std::cout << getKey() << endl;
    return 0;
}

//Note: The output will change according to the date and time.

```

Output

```

24788668
47806121

```

Note: The output will change according to the date and time.

 Comment

More info ▼

Advertise with us

Next Article >

Mid-Square hashing

Similar Reads

DSA Tutorial - Learn Data Structures and Algorithms

DSA (Data Structures and Algorithms) is the study of organizing data efficiently using data structures like arrays, stacks, and trees, paired with step-by-step procedures (or algorithms) to solve problems...

🕒 7 min read

Quick Sort

QuickSort is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted...

🕒 12 min read

Merge Sort - Data Structure and Algorithms Tutorials

Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the divide-and-conquer approach. It works by recursively dividing the input array into two halves, recursively sorting th...

🕒 14 min read

Bubble Sort Algorithm

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst...

🕒 8 min read

Data Structures Tutorial

Data structures are the fundamental building blocks of computer programming. They define how data is organized, stored, and manipulated within a program. Understanding data structures is very important f...

🕒 2 min read

Breadth First Search or BFS for a Graph

Given a undirected graph represented by an adjacency list adj, where each adj[i] represents the list of vertices connected to vertex i. Perform a Breadth First Search (BFS) traversal starting from vertex 0,...

🕒 15+ min read

Binary Search Algorithm - Iterative and Recursive Implementation

Binary Search Algorithm is a searching algorithm used in a sorted array by repeatedly dividing the search interval in half. The idea of binary search is to use the information that the array is sorted and reduce the...

🕒 15 min read

Insertion Sort Algorithm

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. It is like sorting playing cards in your hands. Yo...

🕒 9 min read

Dijkstra's Algorithm to find Shortest Paths from a Source to all

Given a weighted undirected graph represented as an edge list and a source vertex `src`, find the shortest path distances from the source vertex to all other vertices in the graph. The graph contains V vertices,...

🕒 12 min read

Selection Sort

Selection Sort is a comparison-based sorting algorithm. It sorts an array by repeatedly selecting the smallest (or largest) element from the unsorted portion and swapping it with the first unsorted element...

🕒 8 min read
